

Customer Churn Prediction using Machine Learning

Task 03 – AI / Machine Learning

1. Project Overview

This project focuses on predicting whether a customer is likely to churn (leave) or stay with a company. A customer churn prediction system can help businesses identify customers who may leave and take preventive actions to improve customer retention. The project implements multiple classification models, compares their performance, applies cross-validation and hyperparameter tuning, analyzes feature importance, and saves the final trained model using Joblib.

2. Objectives

- Prepare and clean the customer churn dataset.
- Perform feature engineering and feature selection.
- Encode categorical variables.
- Scale feature values.
- Train multiple machine learning classification models.
- Evaluate models using Accuracy, Precision, Recall, F1 Score, and confusion matrices.
- Apply 5-Fold Cross-Validation.
- Perform hyperparameter tuning using GridSearchCV.
- Analyze feature importance.
- Select and save the best-performing model.
- Load the saved model and verify its performance.

3. Dataset

The project uses a customer churn dataset containing 10,000 customer records and 14 original columns. The dataset includes customer information such as Credit Score, Age, Tenure, Balance, Number of Products, Credit Card status, Active Member status, Estimated Salary, Geography, Gender, and customer churn status. The dataset satisfies the minimum 3,000-record requirement.

4. Data Preparation

The preprocessing pipeline included dataset loading, missing-value checking, duplicate checking, feature engineering, feature selection, categorical encoding, feature scaling, and train/test splitting.

5. Feature Engineering

A new feature named **BalanceSalaryRatio** was created to represent the relationship between customer balance and estimated salary. The calculation used the balance divided by estimated salary plus one to avoid division by zero.

6. Feature Selection and Encoding

Irrelevant identifier and unnecessary columns were removed before model training. Geography and Gender were converted into numerical features using One-Hot Encoding with drop_first=True. The final encoded dataset contained $10,000 \times 12$ features.

7. Feature Scaling and Data Split

StandardScaler was used to standardize the feature values. The dataset was split using an 80/20 stratified split: 8,000 records for training and 2,000 records for testing. random_state=42 was used for reproducibility.

8. Machine Learning Models

Four initial classification models were trained: Random Forest, Support Vector Machine (SVM), Gradient Boosting, and Decision Tree. Gradient Boosting was selected for hyperparameter tuning because it achieved the highest mean cross-validation accuracy among the initial models.

9. Final Model Evaluation

Model	Accuracy	Precision	Recall	F1 Score
Random Forest	86.65%	80.17%	45.70%	58.22%
SVM	85.95%	83.87%	38.33%	52.61%
Gradient Boosting	86.95%	78.52%	49.39%	60.63%
Decision Tree	79.35%	49.31%	52.33%	50.77%
Tuned Gradient Boosting	87.10%	82.25%	46.68%	59.56%

The Tuned Gradient Boosting model achieved the highest test accuracy of 87.10%. The original Gradient Boosting model achieved the highest F1 Score among the initial models at 60.63%. SVM achieved the highest precision among the initial models at 83.87%.

10. Confusion Matrix

Confusion matrices were generated to analyze correct and incorrect predictions. For the Gradient Boosting model, the confusion matrix was:

```
[[1538  55]
 [ 206 201]]
```

This means 1538 customers were correctly predicted as staying, 55 were incorrectly predicted as churned, 206 churned customers were missed, and 201 churned customers were correctly identified.

11. Cross-Validation

Model	Mean CV Accuracy	Std. Deviation
Random Forest	86.18%	0.67%
SVM	85.49%	0.62%
Gradient Boosting	86.20%	0.88%
Decision Tree	78.81%	0.40%

Gradient Boosting achieved the highest mean cross-validation accuracy among the initial models at 86.20%.

12. Hyperparameter Tuning

GridSearchCV was used to optimize the Gradient Boosting classifier. The tested parameters were n_estimators, learning_rate, and max_depth. A total of 27 parameter combinations were evaluated using 5-Fold Cross-Validation.

Best Parameters:

learning_rate = 0.05

max_depth = 4

n_estimators = 100

Best Cross-Validation Accuracy: 86.40%

Tuned Test Accuracy: 87.10%

13. Feature Importance

Rank	Feature	Importance
1	Age	38.60%
2	NumOfProducts	28.56%
3	IsActiveMember	12.27%
4	Geography_Germany	6.56%
5	Balance	4.85%
6	BalanceSalaryRatio	3.46%
7	EstimatedSalary	2.02%
8	CreditScore	1.98%
9	Gender_Male	1.23%
10	Tenure	0.33%
11	Geography_Spain	0.08%
12	HasCrCard	0.06%

Age, Number of Products, and Active Member status were the most influential features for the trained Gradient Boosting model.

14. Best Model

Tuned Gradient Boosting Classifier

Accuracy: 87.10%

Precision: 82.25%

Recall: 46.68%

F1 Score: 59.56%

Best Cross-Validation Accuracy: 86.40%

15. Model Saving and Loading

The final model was saved using Joblib as **models/best_model.joblib**. The saved model was loaded again and tested successfully, producing the same test accuracy of 87.10%. This confirms that the trained model can be reused without retraining.

16. Project Structure

Task_03_AI_ML/

■■■ dataset/churn.csv

■■■ models/best_model.joblib

■■■ screenshots/

■■■ main.py

■■■ requirements.txt

■■■ README.md

17. How to Run

1. Open the Task_03_AI_ML folder in VS Code.
2. Install dependencies with: **pip install -r requirements.txt**
3. Run the pipeline with: **python main.py**

The script performs preprocessing, feature engineering, feature selection, encoding, scaling, train/test splitting, model training, evaluation, confusion matrices, cross-validation, hyperparameter tuning, feature importance analysis, model comparison, model saving, and saved-model verification.

18. Limitations

- The model is trained on a single customer churn dataset.
- Performance may change on different datasets.
- Recall for churned customers is lower than overall accuracy.
- The model should be validated using real-world business data before production deployment.
- Additional techniques may be required to improve churn detection.

19. Future Improvements

- Test additional models such as XGBoost.
- Apply class-imbalance handling techniques.
- Improve churn recall.
- Add probability-based churn prediction.
- Build an interactive Streamlit interface.
- Deploy the model as an API.
- Monitor model performance after deployment.
- Add real-time customer churn prediction.

20. Technologies Used

Python • Pandas • NumPy • Scikit-learn • Matplotlib • Joblib

21. Project Status

Task 03 – Completed ■

The complete customer churn prediction machine learning pipeline was implemented, optimized, evaluated, and documented. The final Tuned Gradient Boosting model achieved 87.10% test accuracy and was successfully saved and loaded using Joblib.